Learning Objectives

Learners will be able to...

- **Define and differentiate depth-first (DFS) and breadth-first search (BFS)**
- **Implement depth-first and breadth-first search**
- **Analyze time complexities of DFS and BFS**
- **Define use cases for DFS and BFS**

Tree Searching Algorithms

Finding Nodes in a Tree

In contrast to linear data structures like arrays, stacks, and queues, where data elements are arranged sequentially, trees are non-linear structures where data elements are organized hierarchically. This hierarchical arrangement requires unique searching methods, tailored to traverse the tree in specific orders to access desired nodes.

When it comes to tree-based data structures, the approach to searching is vital in accessing and manipulating data effectively. Two prominent strategies that govern the search of tree structures are **depth-first search (DFS)** and **breadth-first search (BFS)**. In this assignment, we will delve deeper into these two fundamental searching techniques, focusing on their conceptual differences, implementations, and specific scenarios where one may be more suitable than the other. This a very common interview topic, we will be very thorough.

Depth-First Search

DFS is a technique that explores as far as possible along each branch before backtracking. It uses a stack to remember to get back to the branching point when a dead end is reached. In the context of tree structures, it traverses the depth (height) of the tree before exploring siblings. The traversals pre-order, in-order, and post-order are examples of DFS.

▼ **Where is the stack?**
When we covered pre-order, in-order, and post-order traversal, we did not explicitly create a stack ADT to traverse the tree. However, we did use recursion. Every time a recursive method is called, the computer's stack is invoked. So while DFS uses a stack, we do not have to create and manage it.

If we were to perform a DFS on the tree below, the order would depend on the traversal strategy used.

- Pre-order: 5 -> 3 -> 1 -> 4 -> 8 -> 7 -> 9
- In-order: 1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 9
- Post-order: 1 -> 4 -> 3 -> 7 -> 9 -> 8 -> 5



DFS is more suitable when you want to visit every node in the tree and/or when the tree is deep and the solutions are not close to the root.

Breadth-First Search

BFS (sometimes referred to as breadth-first traversal or level-order traversal), on the other hand, traverses level by level. Starting from the root node, it visits all the nodes at the current level from left to right before progressing to the next level. BFS uses a queue data structure to keep track of the nodes at the current level.

If we were to perform a BFS on the tree below, the order would be:

- 5 -> 3 -> 8 -> 1 -> 4 -> 7 -> 9

```
      5
    /   \
   3     8
  / \   / \
 1   4 7   9
```

BFS is more suitable when the tree is wide and the solutions are close to the root and/or you want to find the shortest path or the least number of moves to reach a goal.

```
      5
    /   \
   3     8
  / \   / \
 1   4 7   9
```

1. **Breadth in a Tree (Tree Breadth):** Breadth in a tree, often referred to as “breadth-first traversal” or “level-order traversal,” is a tree traversal algorithm that explores all the nodes at a particular level before moving on to the next level. Starting from the root node, it visits all the nodes at the current level from left to right before progressing to the next level. For the provided tree, a breadth-first traversal would visit the nodes in the order 5, 3, 8, 1, 4, 7, 9.
2. **Depth in a Tree (Tree Depth):** Depth in a tree refers to the distance between the root node and a specific node within the tree. It represents the number of edges in the longest path from the root node to the target node. In the given tree, the depth of node 7 is 2, as it takes two edges to reach node 7 from the root node (5 -> 8 -> 7).

C++ Implementation

Implementing DFS and BFS

To practically understand the workings and differences between DFS and BFS, let's consider a binary tree:



Coding the Tree

To represent the tree, we define the Node and Tree classes.

```
class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int item) {
        data = item;
        left = right = nullptr;
    }
};

class Tree {
public:
    Node* root;

    Tree() {
        root = nullptr;
    }

    void insert(int value) {
        root = insert(root, value);
    }

private:
    Node* insert(Node* node, int value) {
        if (node == nullptr) {
            return new Node(value);
        }

        if (value < node->data) {
            node->left = insert(node->left, value);
        } else if (value > node->data) {
            node->right = insert(node->right, value);
        }

        return node;
    }
};
```

After creating the classes, the button below to compile your code. You should not see any errors.

Initializing the Tree

In the `main` method at the bottom of the editor. Create a `Tree` object and add the values `5`, `3`, `8`, `1`, `4`, `7`, and `9` as node values. Even though they are not yet defined, we are going to call the `dfs` and `bfs` methods.

```
int main() {
    Tree tree;
    tree.insert(5);
    tree.insert(3);
    tree.insert(8);
    tree.insert(1);
    tree.insert(4);
    tree.insert(7);
    tree.insert(9);

    // Perform depth-first search
    std::cout << "DFS traversal:" << std::endl;
    tree.dfs();
    std::cout << std::endl;

    // Perform breadth-first search
    std::cout << "BFS traversal:" << std::endl;
    tree.bfs();
    std::cout << std::endl;

    return 0;
}
```

Depth-First Search (DFS)

In DFS, we start the traversal from the root and go as deep as possible down one branch before backtracking. We are going to use a recursive approach for our DFS implementation. That means the public method takes no arguments and calls the recursive method with the root node. The recursive method will perform a pre-order traversal.

Here's how you can implement DFS using recursion in C++:

```
public:
    void dfs() {
        dfs(root);
    }

private:
    void dfs(Node* node) {
        if (node == nullptr) {
            return;
        }

        std::cout << node->data << " ";
        dfs(node->left);
        dfs(node->right);
    }
```

Breadth-First Search (BFS)

Our BFS implementation explores all the nodes at the current depth before moving on to nodes at the next depth level. To make this more simple, we are going to use C++'s built-in queue data structure. The root node is added (the push method) to the queue. As long as the queue is not empty, remove the node (the pop method) from the queue and print it. Then check to see if there are child nodes. If so, add them to the queue.

```
public:
    void bfs() {
        if (root == nullptr) {
            return;
        }

        std::queue<Node*> queue;
        queue.push(root);

        while (!queue.empty()) {
            Node* node = queue.front();
            queue.pop();

            std::cout << node->data << " ";

            if (node->left != nullptr) {
                queue.push(node->left);
            }

            if (node->right != nullptr) {
                queue.push(node->right);
            }
        }
    }
```

Now that both algorithms have been implemented, run your code. You should see the following output:

```
DFS traversal:
5 3 1 4 8 7 9
BFS traversal:
5 3 8 1 4 7 9
```

▼ Code

> Your code should look like this:
>
> ```
> #include <iostream>
> #include <queue>
>
> class Node {
> public:
>     int data;
>     Node* left;
>     Node* right;
> ```

```
    Node(int item) {
        data = item;
        left = right = nullptr;
    }
};

class Tree {
public:
    Node* root;

    Tree() {
        root = nullptr;
    }

    void insert(int value) {
        root = insert(root, value);
    }

    void dfs() {
        dfs(root);
    }

    void bfs() {
        if (root == nullptr) {
            return;
        }

        std::queue<Node*> queue;
        queue.push(root);

        while (!queue.empty()) {
            Node* node = queue.front();
            queue.pop();

            std::cout << node->data << " ";

            if (node->left != nullptr) {
                queue.push(node->left);
            }

            if (node->right != nullptr) {
                queue.push(node->right);
            }
        }
    }

private:
    Node* insert(Node* node, int value) {
        if (node == nullptr) {
            return new Node(value);
        }

        if (value < node->data) {
            node->left = insert(node->left, value);
        } else if (value > node->data) {
            node->right = insert(node->right, value);
        }
```

```
        return node;
    }

    void dfs(Node* node) {
        if (node == nullptr) {
            return;
        }

        std::cout << node->data << " ";
        dfs(node->left);
        dfs(node->right);
    }
};

int main() {
    Tree tree;
    tree.insert(5);
    tree.insert(3);
    tree.insert(8);
    tree.insert(1);
    tree.insert(4);
    tree.insert(7);
    tree.insert(9);

    // Perform depth-first search
    std::cout << "DFS traversal:" << std::endl;
    tree.dfs();
    std::cout << std::endl;

    // Perform breadth-first search
    std::cout << "BFS traversal:" << std::endl;
    tree.bfs();
    std::cout << std::endl;

    return 0;
}
```

Applying DFS

Depth-First Search (DFS) traverses to the deepest node of a branch before backtracking and exploring other branches. The primary use of DFS is to traverse all nodes of a tree or a graph. In the context of trees, DFS is versatile and can be implemented recursively or using a stack. It's particularly useful when solutions are found in the depth of the tree, for instance, in decision trees.

- **Time Complexity:** DFS has a time complexity of $O(n)$, where n is the number of nodes. Every node is visited once.
- **Space Complexity:** The space complexity of DFS is $O(h)$, where h is the height of the tree. This accounts for the storage of intermediate nodes in the call stack during recursive function calls.

DFS Example Scenarios

1. **Finding a Path:** DFS can be used to find a path between two nodes in a tree. If a path exists, DFS would effectively be able to trace it.
2. **Finding Connected Components:** In a binary tree, DFS can help identify all connected components or nodes reachable from a given node.
3. **Solution Space Exploration:** DFS is often used in scenarios where we need to explore the solution space exhaustively, like solving puzzles or making decisions.

Example Questions

It is not uncommon for interviews to pose technical questions to prospective employees. Depth-first search is often among these questions. Here are some examples of questions that use depth-first search.

1. **Find the Maximum Depth of a Binary Tree:**
   Implement a DFS algorithm to find the maximum depth of a given binary tree. In this example, you are not going to print anything. Instead, you are going to recursively calculate the depths of the left and right subtrees. Then, you return the larger of the two numbers. Remember to add one to take into account the root node of the tree.

```cpp
int maxDepth(Node* root) {
    if (root == nullptr) return 0;
    int leftDepth = maxDepth(root->left);
    int rightDepth = maxDepth(root->right);
    return std::max(leftDepth, rightDepth) + 1;
}
```

2. **Check the Existence of a Path with a Given Sum:**

Given a binary tree and an integer sum, determine if the tree has a root-to-leaf path such that adding up all the numbers along the path equals the given sum. This is an ideal use case for DFS because a depth-first search touches on all nodes along a given path. BFS, however, only cares about nodes in a given level; it cannot determine if there is a valid path between these nodes. This method recursively traverses the left and right subtrees looking for a root-to-leaf path with a given sum. You decrement the sum by the value of the node since this value is now included in the path.

```
bool hasPathSum(Node* root, int sum) {
    if (root == nullptr) return false;
    if (root->left == nullptr && root->right == nullptr) return
        sum == root->data;
    return hasPathSum(root->left, sum - root->data) ||
        hasPathSum(root->right, sum - root->data);
}
```

3. **Find the Lowest Common Ancestor in a Binary Tree:**

Implement a DFS algorithm to find the lowest common ancestor of two given nodes in a binary tree. The lowest common ancestor is the deepest node that has both p and q as descendant nodes in a tree. The method performs a post-order traversal searching for p and q in the left and right subtrees.

```
Node* findLCA(Node* root, Node* p, Node* q) {
    if (root == nullptr || root == p || root == q) return root;
    Node* left = findLCA(root->left, p, q);
    Node* right = findLCA(root->right, p, q);
    if (left != nullptr && right != nullptr) return root;
    return left != nullptr ? left : right;
}
```

In these examples, we see the utility of DFS in solving various problems related to binary trees. By exploring each branch to its deepest node before backtracking, DFS allows for thorough exploration of the solution space in different scenarios. In the next section, we will discuss Breadth-First Search (BFS) and see how it contrasts with DFS in solving different kinds of problems.

Applying BFS

Breadth-First Search (BFS) operates by exploring each level of a tree before proceeding to the next one. It is commonly implemented using a queue, which systematically stores and explores nodes level by level. In the context of trees, BFS is highly effective for locating the shortest path or the nearest item.

- **Time Complexity:** The time complexity of BFS is $O(n)$, where n is the number of nodes, as it visits every node once.
- **Space Complexity:** The space complexity of BFS is $O(w)$, where w is the maximum width (or the maximum number of nodes at a level) of the tree, representing the maximum nodes that can be stored in the queue.

BFS Example Scenarios:

1. **Finding the Shortest Path:** BFS is advantageous for finding the shortest path in unweighted graphs or trees, as it explores all nodes at the current depth before moving to nodes at the next depth level.
2. **Level Order Traversal:** BFS is naturally suited for traversing a tree level by level, which is crucial for various applications, including balancing trees and visual representations.

Example Questions:

1. **Level Order Traversal of a Binary Tree:**
   Implement a BFS algorithm to traverse a binary tree level by level and print nodes at each level. This is the most obvious use case for BFS. You cannot represent the hierarchical structure of a tree with a DFS.

```
void levelOrderTraversal(Node* root) {
    if (root == nullptr) return;
    std::queue<Node*> queue;
    queue.push(root);
    while (!queue.empty()) {
        Node* current = queue.front();
        queue.pop();
        std::cout << current->data << " ";
        if (current->left != nullptr) queue.push(current->left);
        if (current->right != nullptr) queue.push(current-
        >right);
    }
}
```

2. **Minimum Depth of Binary Tree:**
   Use BFS to find the minimum depth of a binary tree. That is, what is the depth of the shallowest leaf node? A BFS works best here because the

question is not asking about relationships between nodes. Instead, this question deals with the general structure of the tree itself. Just as before, this algorithm uses a Queue data structure. It works its way down the levels of the tree looking for the first node that has no children.

```
int minDepth(Node* root) {
    if (root == nullptr) return 0;
    std::queue<Node*> queue;
    queue.push(root);
    int depth = 1;
    while (!queue.empty()) {
        int levelSize = queue.size();
        for (int i = 0; i < levelSize; i++) {
            Node* current = queue.front();
            queue.pop();
            if (current->left == nullptr && current->right ==
        nullptr) return depth;
            if (current->left != nullptr) queue.push(current-
        >left);
            if (current->right != nullptr) queue.push(current-
        >right);
        }
        depth++;
    }
    return depth;
}
```

These examples illustrate how BFS provides solutions by exploring nodes breadth-wise or level by level. It is especially efficient in situations where the solution or required node is expected to be located close to the root or in scenarios where exploring all nodes at a particular depth is beneficial.

Comparing DFS and BFS

The choice between DFS and BFS isn't about "which is better" in a global sense, but "which is more suitable" for the given problem context. By understanding the details highlighted above, we can make informed decisions on the correct algorithm to use based on our specific needs, thereby optimizing performance and efficiency.

Comparison between DFS and BFS

While both DFS and BFS are techniques used for traversing and searching data in trees (and graphs which we will cover later), they have distinct characteristics and performance implications:

1. **Traversal Pattern:**
    - **DFS:** A depth-oriented approach as it explores as far down a branch as possible before backtracking.
    - **BFS:** A breadth-oriented approach, exploring all neighboring nodes at the present depth before moving on to nodes at the next depth level.
2. **Data Structure Used:**
    - **DFS:** Typically uses a stack to keep track of vertices to visit next. It can be implemented recursively with the system call stack or iteratively with a manual stack.
    - **BFS:** Utilizes a queue to keep track of the next vertices to visit, ensuring that vertices are explored in the order they were encountered.
3. **Memory Usage**
    - **DFS:** Might consume less memory than BFS, as it's not necessary to store all child pointers at each level.
    - **BFS:** Can be more memory-intensive, especially when the tree's breadth grows exponentially, as it stores all child pointers at the current level.
4. **Time Complexity**
    - Both DFS and BFS have a time complexity of $O(n)$, where n is the number of nodes, as each node is visited once.
5. **Space Complexity**
    - **DFS:** $O(h)$, where h is the maximum height of the tree.
    - **BFS:** $O(w)$, where w is the maximum width of the tree.

6. **Use-Cases**
    - **DFS:** More suitable when deep solutions are more desirable or to visit every node in the structure. It's also beneficial in scenarios such as solving mazes.
    - **BFS:** Ideal for finding the shortest path on unweighted trees or graphs, or when the solution is not deep, or is closer to the root.
7. **Path Finding**
    - **DFS:** Might not find the shortest path in a tree/graph, especially if a solution is found deep in the structure.
    - **BFS:** Always finds the shortest path (based on the number of edges) in an unweighted graph or tree.
8. **Implementation:**
    - **DFS:** Tends to be a bit more straightforward to implement, especially using recursion.
    - **BFS:** Generally requires an understanding of the queue data structure and the iteration over each level of depth.

Suppose we need to find if a path exists between two nodes in a tree. DFS will start from the root and explore as far as possible along each branch before backtracking. Thus, it's more suitable for scenarios where we need to validate the existence of a path, not necessarily determine the shortest path. On the other hand, BFS starts from the root and explores all neighboring nodes at the present depth before moving on to nodes at the next level. It is better suited for finding the shortest path or evaluating the closest node/condition.

Formative Assessment 1

Formative Assessment 2